CONVERGE

Composed alpha release guide
Version 0.2.0-alpha.1

SEAMWISE	TASK-SPEC	CONVERGE
reviewed decomposition	task authority	coordination + settlement

Release candidate: local evidence is documented; hosted CI and publication remain explicit gates.

Converge

Coordinate intent, decomposition, task authority, execution, and settlement without duplicating authority.

Converge 0.2.0-alpha.1 | cvg 0.2.0-alpha.1 | Seamwise 0.2.0-alpha.1 | Task-Spec 3.8.0

Install · First composed journey · Authority model · Composed flow · CLI reference

Release truth

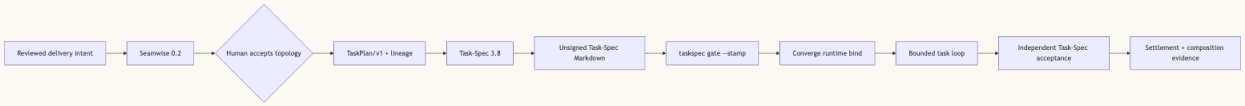
This checkout is the Converge 0.2.0-alpha.1 release candidate. The composed implementation, deterministic cross-engine tests, and authenticated Codex demo are locally verified against the exact candidate commits. Publication is not complete until the exact Task-Spec, Seamwise, and Converge commits pass hosted macOS and Linux CI and are tagged in dependency order.

Claim	Current evidence
Task-Spec 3.8.0 implementation	Local `CHECK=READY`, `CONFORMANCE=L2`; draft PR on commit `44c23e974b6448cdaf21e7514f297a757154beac`
Seamwise 0.2.0-alpha.1 implementation	Local `RELEASE=READY`; draft PR on commit `29a087c63ee97cb07ab4635aaba87ecab48dc2f1`
Converge 0.2.0-alpha.1 implementation	Local `release-check` green; 57 forms / 227 JSON calls; authenticated Codex settled and accepted on the exact engine candidates
Hosted CI	Blocked before execution while GitHub jobs report zero steps; sibling private-repository jobs also require a scoped `RELEASE_STACK_READ_TOKEN`
Published tags	The new three-repository stack is not published until hosted gates pass
Historical Converge 0.1.0	Published and immutable; it documents the former bundled Task-Spec architecture

The release never retags or moves `v0.1.0`. See alpha readiness for the live gate ledger and release notes for migration details.

What Converge owns

Converge is the thin coordinator and assurance layer around two independent engines. It calls their public binaries. It does not import, vendor, or copy their implementations.



Rendered from canonical Mermaid source.

System	Sole authority
Seamwise	Evidence-backed seams, swimlanes, capability legs, reviewed decomposition, `TaskPlan/v1`, and lineage
Task-Spec	TaskPlan validation, materialization, Task-Spec structure, authorization, handoff, evals, and acceptance
Converge	Cross-engine sequencing, executable binding, bounded execution, settlement, and composition receipts
Human reviewer	Acceptance of Seamwise topology and explicit risk decisions
Executor	Product-code changes inside the authorized runtime contract; never self-acceptance

Duplicate capability is tolerable. Duplicate authority is not. A Seamwise review does not authorize task dispatch, a Task-Spec materialization receipt does not sign a task, and model narration is never settlement evidence.

Install

Requirements

- Git
- Bash 3.2 or newer
- Python 3
- Task-Spec 3.8.0 for every Converge installation
- Seamwise 0.2.0-alpha.1 only for `cvg compose`
- Node 22 only for the npm door and Cockpit

After the tags are published, install in dependency order:

```
git clone --branch v3.8.0 https://github.com/luanmorenommaciel/task-spec.git
bash task-spec/install.sh --global --copy
taskspec demo

python3 -m pip install   "git+https://github.com/luanmorenommaciel/seamwise.git@v0.2.0-alpha.1"

git clone --branch v0.2.0-alpha.1   https://github.com/luanmorenommaciel/converge.git
bash converge/install.sh --target /absolute/path/to/your-project --copy
```

Until hosted CI is repaired and the tags exist, use the exact release-candidate commits listed in Release truth. Do not treat a source checkout as a published release.

Converge also supports:

```
npm install -g github:luanmorenomaciel/converge
cvg-install
```

or, after publication:

```
CVG_REF=v0.2.0-alpha.1   bash -c "$(curl -fsSL https://raw.githubusercontent.com/luanmorenomaciel/
converge/main/install.sh)"
```

The installer projects exactly eleven Converge skills to `.agents/skills/`, `.claude/skills/`, and `.grok/skills/`. It installs no Task-Spec or Seamwise implementation. Copy mode pins the coordinator, contracts, templates, and skills into the consumer.

First composed journey

Start from a Git repository whose recipe is committed. Explicit binary overrides make the exact engine candidates auditable.

```
export CVG_TASKSPEC_BIN=/absolute/path/to/task-spec/bin/taskspec
export CVG_SEAMWISE_BIN=/absolute/path/to/seamwise/bin/seamwise

cvg compose prepare --source recipe.yaml
# COMPOSE=NEEDS_REVIEW

cvg compose review   --reviewer "repository-owner"   --reason "The seams, ownership, dependencies, and
rollback paths are accepted."
# COMPOSE=PREVIEW_READY

cvg compose preview
# COMPOSE=PREVIEW_READY

cvg compose materialize
# COMPOSE=MATERIALIZED

cvg compose status
# COMPOSE=MATERIALIZED
```

The materialized leaves still contain `signed_off: false`. Authorization remains explicit and per leaf:

```
taskspec gate --stamp cvg/tasks/T-20260815-health-status.md
cvg bind --task cvg/tasks/T-20260815-health-status.md
git add cvg/tasks cvg/execution
git commit -m "authorize and bind health status task"
cvg loop --issue T-20260815-health-status --agent codex
```

A successful loop must end in `TASK_LOOP=LOCAL_SETTLED` or `TASK_LOOP=SETTLED` and `ACCEPTED=1`. The composition receipt is stored at `cvg/receipts/composition/composition-receipt.json`; it binds engine versions, the immutable source commit, Seamwise review/lineage/TaskPlan digests, Task-Spec materialization evidence, and every task hash. It always records `dispatch_authorized: false`.

Existing direct flow

The established nine-pass flow remains available. Task lifecycle verbs are thin delegations to the standalone Task-Spec engine.

```
cvg init
cvg setup signing
cvg lane "add a health endpoint"

cvg capture
cvg intent
cvg structure
cvg decompose
cvg review --adversary codex
cvg review --check

cvg tasks plan
cvg tasks new add-health-endpoint S
cvg tasks validate cvg/tasks/T-20260815-health-status.md
cvg tasks gate --stamp cvg/tasks/T-20260815-health-status.md
cvg bind --task cvg/tasks/T-20260815-health-status.md
cvg loop --issue T-20260815-health-status --agent codex
```

The task backlog lives under `cvg/tasks/`. Converge exports `TASKSPEC_WORKSPACE_ROOT`, `TASKSPEC_BACKLOG_DIR`, and `TASKSPEC_ACCEPTANCE_DIR` to keep nested workspaces on the same explicit root.

Machine contract

Every public form accepts global `--json` and `--dry-run` in any position.

```
cvg --json help
cvg version --json
cvg agent-context --json
cvg compose --json status
```

`--json` emits one `ConvergeCLIResult/v1` document, preserves the underlying exit code, emits no ANSI, and reports `changed` and `dry_run`. The canonical 57-form matrix is contracts/cli-command-matrix.json; the human reference and test coverage derive from it.

Stable compose states are:

- `COMPOSE=NEEDS_REVIEW`
- `COMPOSE=PREVIEW_READY`
- `COMPOSE=MATERIALIZED`
- `COMPOSE=BLOCKED`
- `COMPOSE=ENGINE_UNAVAILABLE`

`cvg compose status` is read-only and returns one safe next action. It blocks on a stale review, changed plan, mismatched task set, changed task bytes, incompatible engine, or stale receipt.

Cockpit

Cockpit is a read-only observation and interpretation surface over `cvg snapshot`. It does not become a second source of truth and it cannot authorize work.

```
npm run cockpit:install
npm run cockpit:dev --   --cvg-home "$PWD"   --project-root /absolute/path/to/project
```

Ask Converge is optional ACP interpretation. Agent prose is not a gate verdict, receipt, or acceptance record.

Repository map

Path	Role
`bin/`	Stable CLI plus focused private helpers
`contracts/`	Canonical CLI matrix and versioned JSON Schemas
`skills/`	Exactly eleven Converge orchestration and assurance skills
`apps/cockpit/`	Read-only observer UI
`templates/`	Consumer workspace templates
`tests/`	Hermetic gate, install, loop, JSON, and composed-flow suites
`docs/`	Architecture, composed flow, CLI reference, release notes, and archived evidence

Verification

One Makefile owns the release entrypoints:

```
make check
make check-json
make check-docs
make check-composed
make check-live-evidence
make demo-composed
make release-check
```

`release-check` is necessary but not sufficient for publication. Hosted Ubuntu, macOS, Cockpit, JSON, docs, package, and composed-E2E jobs must run on the exact commit. A zero-step GitHub job is infrastructure evidence, not a passed gate.

Scope of this alpha

This alpha promises a reproducible composed single-task path and preserves the existing direct Converge flow. It does not promise Manager fleet scheduling, production reliability, a live tracker, or autonomous approval of human decisions.

Documentation

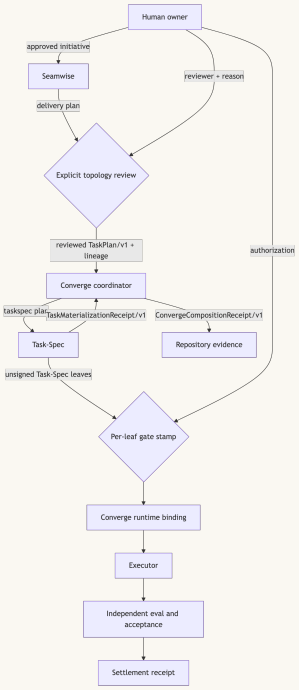
- Architecture and authority
- Composed flow and failure semantics
- CLI reference
- Alpha readiness ledger
- 0.2.0-alpha.1 release notes
- Documentation and archive inventory
- Skill catalog
- Cockpit guide

Converge composed architecture

Status: canonical for Converge 0.2.0-alpha.1.

Architectural invariant

Converge coordinates independently versioned engines over executable and JSON boundaries. It never imports or vendors Seamwise or Task-Spec. Each decision has one authority, and each boundary artifact is digest-bound before the next authority may act.



Rendered from canonical Mermaid source.

Authority table

Decision or artifact	Sole authority	Explicit non-authority
Evidence-backed seams, swimlanes, legs	Seamwise	Converge does not reinterpret topology
Human acceptance of delivery topology	Named reviewer through Seamwise	Neither engine self-approves
`TaskPlan/v1` projection and lineage	Seamwise	Seamwise does not create Task-Spec Markdown
TaskPlan validation and task materialization	Task-Spec	Converge does not render or validate Task-Spec internals
Task dispatch authorization	Task-Spec `gate --stamp`	Materialization and Seamwise review both record false
Runtime contract and path policy	Converge	A Task-Spec cannot widen the repository fence
Product-code mutation	Authorized executor	Coordinator and observer do not write product code
Evals, handoff, acceptance	Task-Spec	Executor narration is not evidence
Cross-engine sequence and composition receipt	Converge	Receipt does not authorize dispatch
Workspace observation	Cockpit	Observer cannot mutate canonical state

Binary boundaries

Converge resolves engines with these explicit overrides:

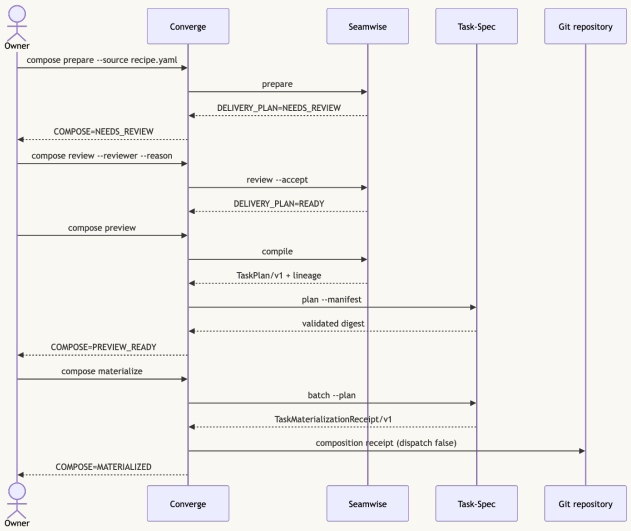
- `CVG_TASKSPEC_BIN`: required for the core Converge task lifecycle and compose.
- `CVG_SEAMWISE_BIN`: required only for `cvg compose`.

Composition first asks Seamwise for `SeamwiseCapabilities/v1` and verifies:

- the supported engine major;
- `SeamwiseCLIResult/v1`;
- `TaskPlan/v1`;
- `SeamwiseTaskPlanLineage/v1`;
- `materializes_tasks: false`;
- `dispatch_authority: false`.

It separately asks Task-Spec for `TaskSpecCLIResult/v1` and requires a version from 3.8.0 inclusive to 4.0.0 exclusive. A missing executable, malformed JSON, exit-code mismatch, incompatible contract, or version change during a command returns `COMPOSE=ENGINE_UNAVAILABLE` and changes no canonical task state.

Artifact flow



Rendered from canonical Mermaid source.

Seamwise compilation is transactional: the only boundary outputs are `seamwise/task-plan.json` and `seamwise/task-plan-lineage.json`. Task-Spec materialization writes leaves under the configured Converge backlog. Converge then writes:

- `cvg/receipts/composition/source.json`;
- `cvg/receipts/composition/taskspec-materialization-receipt.json`;
- `cvg/receipts/composition/composition-receipt.json`.

The final receipt binds all three versions, the committed recipe bytes and Git commit, Seamwise review and lineage file hashes, the TaskPlan digest, Task-Spec materialization receipt hash, and every materialized task hash.

Settlement and observation

After explicit Task-Spec authorization, Converge binds a task-scoped execution profile. The loop mints an attempt-bound handoff, runs a bounded executor, checks the repository path policy, executes declared evals, and asks Task-Spec for independent acceptance before settlement. A successful run provides both:

- `TASK_LOOP=LOCAL_SETTLED` or `TASK_LOOP=SETTLED`;
- `ACCEPTED=1`.

Cockpit reads `WorkspaceSnapshot 3.0`. It may explain evidence, but cannot approve a review, stamp a task, alter a receipt, or change settlement state. Manager fleet scheduling remains outside the alpha scope.

Security and failure posture

- All receipt and task paths must resolve inside the project root.
- Symlinked composition paths are refused.
- The source recipe must be tracked and byte-identical to its Git commit before materialization.
- Existing task bytes must match exactly on rerun; conflicts fail closed.
- Composition status re-hashes the complete chain and blocks stale evidence.
- Materialization interruption is recoverable because Task-Spec reruns are idempotent and the Converge receipt is written last.
- `dispatch_authorized` is always false in materialization and composition receipts.

The versioned schemas under `contracts/` are the machine contracts. This document explains ownership; it does not replace them.

Composed flow

Status: canonical operating guide for Converge 0.2.0-alpha.1.

Before starting

Use a Git repository, commit the Seamwise recipe, and resolve the exact engines:

```
export CVG_TASKSPEC_BIN=/absolute/path/to/task-spec/bin/taskspec
export CVG_SEAMWISE_BIN=/absolute/path/to/seamwise/bin/seamwise
git add recipe.yaml
git commit -m "pin composed delivery recipe"
```

The release pairing is Task-Spec 3.8.0, Seamwise 0.2.0-alpha.1, and Converge 0.2.0-alpha.1. `cvg compose status` is always safe and read-only.

1. Prepare

```
cvg compose prepare --source recipe.yaml
```

Expected state: `COMPOSE=NEEDS_REVIEW`.

Converge asks Seamwise to initialize, map, and plan. It verifies that prepare did not emit a TaskPlan. A changed recipe cannot silently replace an existing mapped source.

2. Review

Read `seamwise/delivery-plan.yaml`, the source evidence, every seam owner, dependency, rollback, and unresolved objection. Then record a real decision:

```
cvg compose review \
  --reviewer "repository-owner" \
  --reason "Ownership, topology, dependencies, and rollback paths are accepted."
```

Expected state: `COMPOSE=PREVIEW_READY`.

Review performs no compilation. The reviewer and reason become part of the hash-bound Seamwise review receipt.

3. Preview

```
cvg compose preview
```

Expected state: `COMPOSE=PREVIEW_READY`.

Converge asks Seamwise for exactly two projections, verifies their lineage, then calls `taskspec plan`. No Task-Spec Markdown is written. A missing review, tampered plan, lineage mismatch, invalid dependency, or contract drift blocks.

4. Materialize

```
cvg compose materialize
```

Expected state: `COMPOSE=MATERIALIZED`.

Task-Spec is the only engine called to materialize leaves. Converge checks the exact TaskPlan digest, task IDs, output paths, and bytes before writing its own composition receipt. Repeating the command is byte-idempotent when the receipt and tasks are current.

5. Authorize each leaf

Materialization is deliberately not authorization:

```
grep '^signed_off:' cvg/tasks/T-20260815-health-status.md
taskspec gate --stamp cvg/tasks/T-20260815-health-status.md
```

The first command must show `signed_off: false`. Only the second may create the Task-Spec authorization seal.

6. Bind

```
cvg bind --task cvg/tasks/T-20260815-health-status.md
git add cvg/tasks cvg/execution
git commit -m "authorize and bind composed task"
```

Expected gate: `CHECK_RUNTIME_CONTRACT=PASS`. Binding freezes the task revision, evidence slice, runtime capabilities, and repository write fence.

7. Loop

```
cvg loop --issue T-20260815-health-status --agent codex
```

The loop never chooses its own work. It executes one assigned task under iteration, time, token, and path limits. An engine error, timeout, exhausted budget, or uncommitted out-of-scope write is not success.

8. Accept and settle

The loop asks Task-Spec to independently accept the exact handoff and task revision. Sign-off requires:

```
TASK_LOOP=LOCAL_SETTLED
ACCEPTED=1
```

`TASK_LOOP=SETTLED` is also valid when external publication is allowed. Model narration, a green-looking diff, or a Converge composition receipt alone is not acceptance evidence.

State and recovery table

State	Meaning	One safe next action
`COMPOSE=BLOCKED`	No prepared plan or evidence failed verification	Follow the `NEXT=` action; never bypass the failed contract
`COMPOSE=NEEDS_REVIEW`	Delivery plan exists without current human acceptance	Review the plan, then run `cvg compose review`
`COMPOSE=PREVIEW_READY`	Review is current; preview or materialization is next	Run the exact `NEXT=` action from status
`COMPOSE=MATERIALIZED`	Tasks and receipts re-hash successfully	Run `taskspec gate --stamp` on one intended leaf
`COMPOSE=ENGINE_UNAVAILABLE`	Binary, version, JSON, or capabilities are incompatible	Install or select the exact supported engine

An interrupted materialization may leave the Task-Spec receipt without the final Converge receipt. Rerun `cvg compose materialize`; Task-Spec proves the existing bytes are unchanged and Converge completes the final binding.

JSON automation

```
cvg compose status --json
cvg --json compose preview
```

Both forms emit one `ConvergeCLIResult/v1` document and preserve the underlying exit code. Do not parse prose; branch on `token`, `exit_code`, `changed`, and `dry_run`.